

Projet Python — Séance 4

Python idiomatique, programmation orientée objet, matplotlib et finalisation

Enseignante : Rahma GARGOURI — 2026-2027

Séance de 3 heures — Projet en groupe

Objectif. Dernière séance. Rendre son code plus **idiotique** (compréhensions, f-strings...), découvrir la **programmation orientée objet** (les classes), produire des **graphiques** avec `matplotlib`, puis **assembler et finaliser** le projet.

Comme précédemment, le travail se fait sur une branche `seance-4`, avec une *pull request* en fin de séance. Après chaque petite partie, une boîte bleue rassemble des questions à **rédigier** dans `docs/notes_seance4.md` et à **commiter**. L'IA est autorisée, mais chaque réponse doit être comprise (elle pourra être demandée à l'oral).

Partie A — Python plus idiomatique

A.1 — Les compréhensions. Réécrire une boucle de la séance 2 (par exemple le calcul d'une liste de variations) sous forme de **compréhension de liste** (`[x*2 for x in liste]`). Faire de même avec une **compréhension de dictionnaire**.

A.2 — `enumerate`, `zip`, `sorted`. Parcourir une liste avec `enumerate` (indice + valeur), associer deux listes avec `zip` (par exemple les dates et les taux), et trier avec `sorted(..., key=...)`.

A.3 — f-strings, `lambda`, `map/filter`. Formater des nombres et des dates avec des **f-strings** (`f"{taux:.2f}"`). Utiliser une fonction `lambda` avec `map` ou `filter`.

Questions — à rédiger et à commiter

1. C'est quoi une **compréhension** de liste ? Quel avantage par rapport à une boucle `for` classique ?
2. À quoi servent `enumerate` et `zip` ?
3. Comment trier une liste selon un critère (`sorted` avec `key`) ?
4. C'est quoi une **f-string** ? Comment afficher un nombre à deux décimales ?
5. C'est quoi une fonction `lambda` ?

Partie B — La programmation orientée objet (les classes)

B.1 — Votre première classe. Créer une classe (par exemple `SerieTaux`) qui regroupe des **données** (les taux d'une devise) et des **méthodes** (moyenne, variation, taux à une date). Comprendre `__init__`, les **attributs**, les **méthodes** et le paramètre `self`.

B.2 — Plusieurs objets. Créer plusieurs **instances** (une par devise). Distinguer un **attribut de classe** (partagé par toutes les instances) d'un **attribut d'instance** (propre à chaque objet). Attention au piège d'une **liste mutable** déclarée en attribut de classe (le même partage que celui vu pour les dictionnaires en séance 1).

B.3 — Les dataclasses. Découvrir `dataclass`, qui évite d'écrire soi-même la méthode `__init__`.

Questions — à rédiger et à commiter

1. C'est quoi une **classe**? Un **objet**? Une **instance**? À quoi sert `self`?
2. À quoi sert `__init__`?
3. Différence entre un attribut de classe et un attribut d'instance?
4. Qu'apporte une `dataclass` par rapport à une classe écrite entièrement à la main?

Partie C — Visualiser avec matplotlib

C.1 — Une première courbe. Installer `matplotlib` (`pip install matplotlib`). Tracer l'évolution d'une devise sur la période (courbe date → taux). Chaque figure a un **titre**, des **axes** **légendés**, et est sauvegardée en **PNG**.

C.2 — Plusieurs graphiques. Comparer les deux devises du groupe sur un même graphique, et tracer un **histogramme** des variations quotidiennes. Utiliser plusieurs graphiques **côte à côte** (subplots).

Questions — à rédiger et à commiter

1. Différence entre la **figure** et les **axes**? Pourquoi préférer `fig, ax = plt.subplots()` à l'usage direct de `pyplot`?
2. Quel type de graphique pour quel type de donnée (série temporelle, distribution, comparaison)?
3. Comment sauvegarder une figure en PNG?

Partie D — Assembler et finaliser

D.1 — La ligne de commande. Relier vos scripts avec `argparse` : une commande unique avec des **sous-commandes** (par exemple `extraire`, `analyser`, `graphiques`). Comprendre `if __name__ == "__main__"` et l'organisation du code en **modules** (dossier `src/`).

D.2 — Le rendu final. Rédiger le **README** (installation + exemples de commandes), figer `requirements.txt`, écrire le **rapport final** (démarche, graphiques, résultats). Ouvrir la **dernière PR**.

Questions — à rédiger et à commiter

1. À quoi sert `argparse`? Pourquoi une ligne de commande plutôt que des `input()`?
2. Que fait exactement `if __name__ == "__main__"`?
3. C'est quoi un **module**? Un **package**?
4. Que doit contenir un bon README? À quoi servent `requirements.txt` et `.gitignore`?

Contraintes imposées

A la fin, le dépôt doit être **complet, propre et exécutable**.

Livrables de la séance (commit + push + PR avant la fin)

Le code rendu plus **idiotique** — la (les) **classe(s)** (par exemple `SerieTaux`) — les **graphiques** PNG — l'outil en **ligne de commande** (`argparse`) — `README.md` et `requirements.txt` à jour — `docs/notes_seance4.md` — **dernière PR** (séance 4) ouverte.